


```
model.to(device)
model.eval()
```

[Show hidden output](#)

```
def preprocess_image(pil_img):
    transform = transforms.Compose([
        transforms.Resize((320, 320)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                              [0.229, 0.224, 0.225])
    ])
    return transform(pil_img).unsqueeze(0)

def load_image(path):
    img = Image.open(path).convert("RGB")
    img_tensor = preprocess_image(img) # use the above function
    return img, img_tensor.to(device)
```

```
import torch
import numpy as np
from PIL import Image

# Inference function
def inference(image_path, save_path="output.png"):
    # Load image and preprocess
    img, img_tensor = load_image(image_path)

    with torch.no_grad():
        # U²-Net outputs multiple side outputs; take the first one
        pred = model(img_tensor)[0]
        pred = pred.squeeze().cpu().numpy() # shape [H, W]

        # Normalize mask to [0, 1]
        pred = (pred - pred.min()) / (pred.max() - pred.min() + 1e-8)

    # Resize mask back to original image size
    mask = Image.fromarray((pred * 255).astype(np.uint8)).resize(img.size)

    # Apply mask to original image (keep foreground, remove background)
    img_rgba = img.convert("RGBA")
    mask = np.array(mask) / 255.0
    img_array = np.array(img_rgba)

    img_array[..., 3] = (mask * 255).astype(np.uint8) # set alpha channel

    result = Image.fromarray(img_array, "RGBA")
    result.save(save_path)
    return result
```

```
output = inference("/kaggle/working/pics/glass.jpeg")
imgggggggggg=Image.open("/kaggle/working/output.png")
imgggggggggg
```



```
imgggggggggg.size
```

```
img_array = np.array(imgggggggggg)
img_array.shape
```

```
(145, 348, 4)
```

